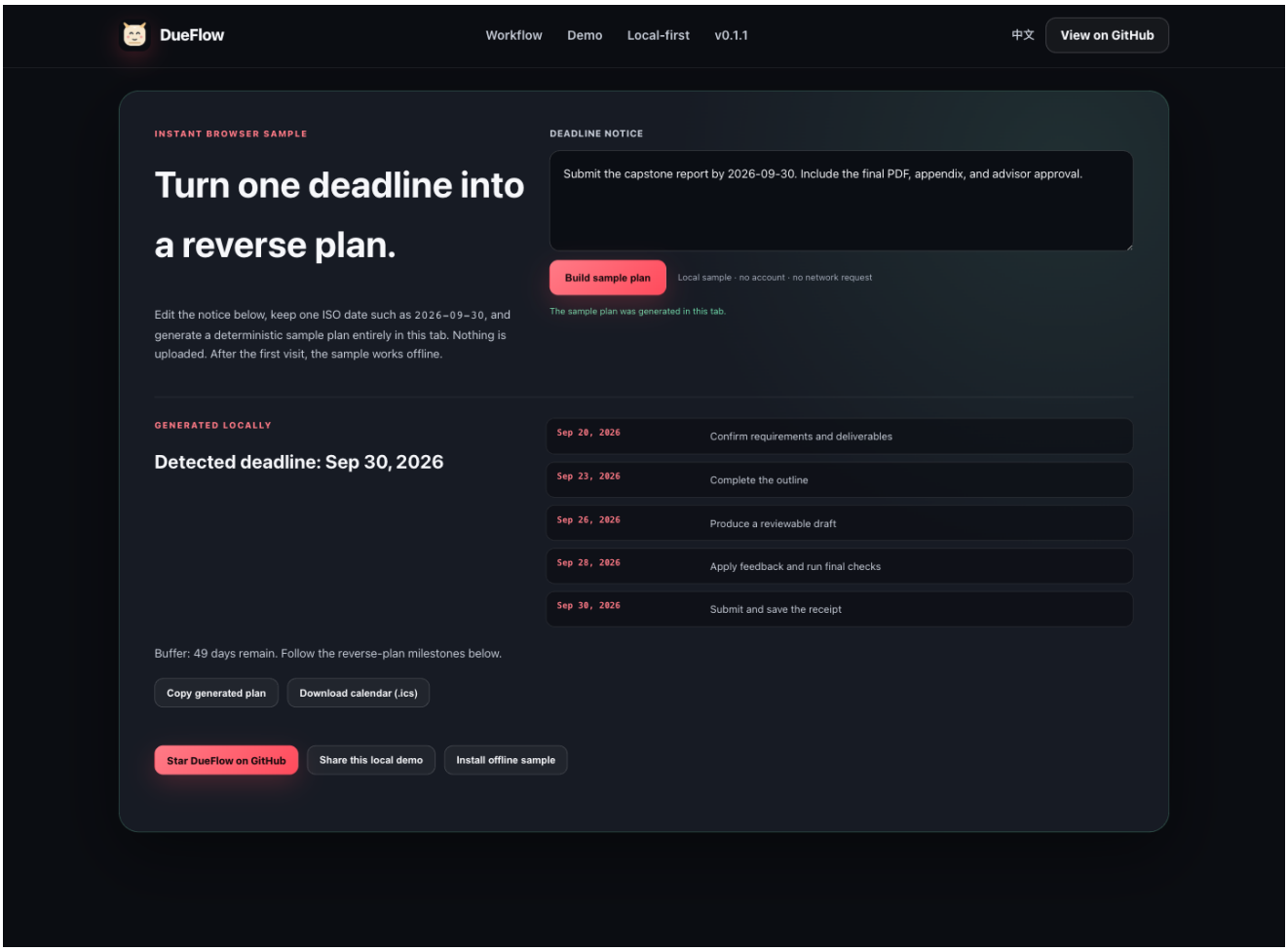
DueFlow

Turn scattered deadline information into an actionable schedule - locally.

Open the 60-second browser sample



After the first visit, the sample also works offline. Copy the plan or download a standard .ics calendar; no signup, API key, or upload is required.

Why DueFlow

What you have	What DueFlow produces
Course notices, internship emails, competition announcements	Structured tasks with deadlines, deliverables, and submission details
Screenshots, PDFs, Markdown, text files, webhook payloads	One deduplicated local Inbox with source references
A deadline and too many unknowns	Reverse plans, missing-information checks, and risk flags
A schedule you need elsewhere	Editable Markdown, todo lists, summaries, and calendar `.ics` exports

Highlights

- Local-first by default — SQLite data, Inbox files, exports, backups, and diagnostics stay on your machine.
- Multiple intake paths — text, Markdown, PDF, OCR-ready images, file drops, local Inbox scanning, and webhooks.
- Actionable extraction — deadlines, deliverables, submission methods, missing information, and source quotes.
- Planning, not just parsing — editable schedule items, reverse plans, risk checks, reminders, and exports.
- Desktop companion — a Tauri 2 shell with a transparent always-on-top pet and a separate schedule surface.
- Reproducible verification — mock-provider tests, desktop smoke tests, backup/restore checks, and release gates.

60-Second Local Demo

```
git clone https://github.com/Ustinian5/DueFlow.git
cd DueFlow
conda env create -f environment.yml
conda run -n dueflow python scripts/run_demo.py
```

Expected result:

```
DueFlow demo completed
processed=3 tasks=3 plans=17 risks=5
```

Generated files appear in `exports/`, including `todo.md`, `plan.md`, `summary.md`, `calendar.ics`, and `submission_report.md`.

Current Status

DueFlow `0.1.4` is ready for local development and open-source review. Native Apple Silicon and Intel developer previews are self-contained zipped `.app` bundles: their loopback-only API sidecar is included, so testers do not need Python or Conda at runtime. The previews have a verified ad-hoc macOS resource seal, but remain unsigned with Developer ID and are not notarized; the distribution boundary is documented in the release guide.

The desktop architecture uses its own Tauri/Python/React implementation. OpenPets is referenced for interaction and architecture research only. DueFlow does not depend on OpenPets and does not import its packages, code, or assets.

Architecture

```
Inputs
  -> Desktop API / Webhook / Streamlit / Desktop Pet
  -> Inbox + deduplication
  -> parser and optional OCR
  -> LLM extraction
  -> automatic SQLite tasks/plans/risks
  -> reminders, exports, diagnostics and desktop pet state
```

Main components:

- `api/desktop.py`: local FastAPI desktop API used by the Tauri frontend.
- `api/webhook.py`: smaller webhook API for external payloads.
- `src/`: core database, parsing, extraction, planning, risk, export and pet-state logic.
- `desktop/`: Tauri 2 + React desktop shell.
- `desktop/src/PetOverlay.tsx`: desktop pet overlay.
- `desktop/src/petRuntime.ts`: event-driven pet state and action runtime.
- `desktop/src/skillRegistry.ts`: local skill manifest validation.
- `desktop/src/petManifest.ts`: local desktop pet appearance validation and runtime resolution.
- `docs/`: desktop API, release process, product plan and OpenPets reference notes.

Requirements

- Python 3.11
- Node.js 20 or newer
- Rust stable toolchain
- Conda is recommended for the Python environment
- macOS for local `.app` packaging

Linux CI or local Rust checks for Tauri may need GTK/WebKit dependencies. See `.github/workflows/test.yml` for the current package list.

Quick Start

Create the Python environment:

```
conda env create -f environment.yml
conda activate dueflow
```

Or install with pip:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

For editable Python package metadata:

```
python -m pip install -e ".[dev]"
```

Create local configuration:

```
cp .env.example .env
```

The default `.env.example` uses:

```
LLM_PROVIDER=mock
DATABASE_PATH=dueflow.db
INBOX_PATH=inbox
EXPORT_PATH=exports
```

Image and screenshot intake requires OCR. On macOS, DueFlow can use the local Vision framework. On other systems, or when overriding the default, set `DUEFLOW_OCR_COMMAND`, for example:

```
DUEFLOW_OCR_COMMAND=tesseract {path} stdout -l chi_sim+eng
```

Run the core tests:

```
python -m pytest
```

Run the command-line demo:

```
python scripts/run_demo.py
```

Desktop Development

Install frontend dependencies:

```
cd desktop
npm install
```

Run the local desktop API:

```
conda run -n dueflow python -m uvicorn api.desktop:app --host 127.0.0.1 --port 8000
```

Run the frontend in browser development mode:

```
cd desktop
npm run dev
```

Run the full Tauri shell:

```
cd desktop
npm run tauri:dev
```

The shell opens:

- `main`: the full DDL workbench.
- `pet`: a transparent always-on-top desktop pet window rendered from `index.html?view=pet`.

The Tauri shell can also autostart the local API. Runtime paths are stored in the platform app data directory instead of the repository.

Verification

Run these before submitting a pull request:

```
python -m pytest

cd desktop
npm run build
npm run test:runtime
npm run test:smoke
npm run test:preflight
npm run test:release-manifest

cd src-tauri
cargo fmt --check
cargo check
cargo test
```

What the gates cover:

- `pytest`: Python pipeline, database, desktop API, packaging metadata and webhook behavior.
- `npm run build`: TypeScript and Vite production build.
- `npm run test:runtime`: pet runtime, event bridge, reminders, local skills and pet manifest logic.
- `npm run test:smoke`: isolated desktop API smoke test with temporary data; validates file intake, duplicate detection, task generation, exports, backup/restore, diagnostics privacy and Tauri window config.
- `cargo test`: Tauri shell helper logic, including local pet import source confinement, asset copying and rollback.

Release

Download the macOS developer preview

Download the native Apple Silicon or Intel `.app.zip` from DueFlow v0.1.4. Each architecture has its own checksum and release manifest; the manifest records the bundled backend hash, packaging-time self-check, and verified ad-hoc resource seal. These developer previews are not Developer ID signed or notarized; use them only if you are comfortable testing an open-source development build.

For local macOS packaging:

```
cd desktop
npm run release:mac
```

The release script builds and self-checks the standalone local API sidecar, runs preflight checks, builds the Tauri app, verifies the `.app` bundle, then writes:

- `desktop/release/DueFlow-Desktop_<version>_<arch>.app.zip`
- `desktop/release/*.sha256`
- `desktop/release/*.manifest.json`

See docs/desktop_release.md for the full release gate, signing and notarization notes. Use docs/release_checklist.md before publishing a GitHub release or handing off a build.

Local Data And Privacy

DueFlow is local-first:

- SQLite database, Inbox files, exports, backups and diagnostics stay on the user's machine.
- Real model providers are optional and configured by the user.

- Diagnostics intentionally omit raw Inbox text, task titles, source quotes and extracted descriptions.
- `.env`, databases, local Inbox files and release artifacts are ignored by Git.

See PRIVACY.md for model-provider, OCR, diagnostics and local asset privacy boundaries.

OpenPets Reference Boundary

OpenPets is used only as a reference sample for:

- event-driven interaction;
- derived pet state;
- action registration;
- plugin/manifest isolation;
- separation between pet window, control surface and desktop shell.

DueFlow does not import OpenPets packages, copy its source, reuse its assets, or adopt its Electron runtime. See docs/openpets_reference_notes.md.

Documentation

- Current product requirements
- Desktop API
- Desktop release
- Release checklist
- Changelog
- Roadmap
- Contributing
- Code of conduct
- Support
- Privacy
- Security policy
- Current product requirements
- Demo notes
- OpenPets reference notes

Contributing

Contributions are welcome. Start with CONTRIBUTING.md, run the verification gates above, and avoid committing private data or generated local artifacts.

Support

Use SUPPORT.md for bug reports, feature requests and support boundaries.

Code of Conduct

Participation in DueFlow project spaces follows CODE_OF_CONDUCT.md.

Security

Please report vulnerabilities privately. See SECURITY.md.

License

DueFlow is released under the MIT License. See LICENSE.